Dynamic programming (DP) is a method for solving complex problems by breaking them into smaller, overlapping sub-problems and storing their results to avoid repeated work.

Instead of recomputing the same values many times (as in naive recursion), DP solves each sub-problem once and reuses the answer. This makes it much more efficient, often reducing exponential time algorithms to polynomial time.

Key ideas:

- **Overlapping sub-problems**: the same smaller problems appear multiple times
- **Optimal sub-structure**: the overall optimal solution can be built from optimal solutions of sub-problems
- **Memoization (top-down)**: store results during iterations
- **Tabulation (bottom-up)**: build solutions iteratively from smallest cases

Simple example: Fibonacci sequence

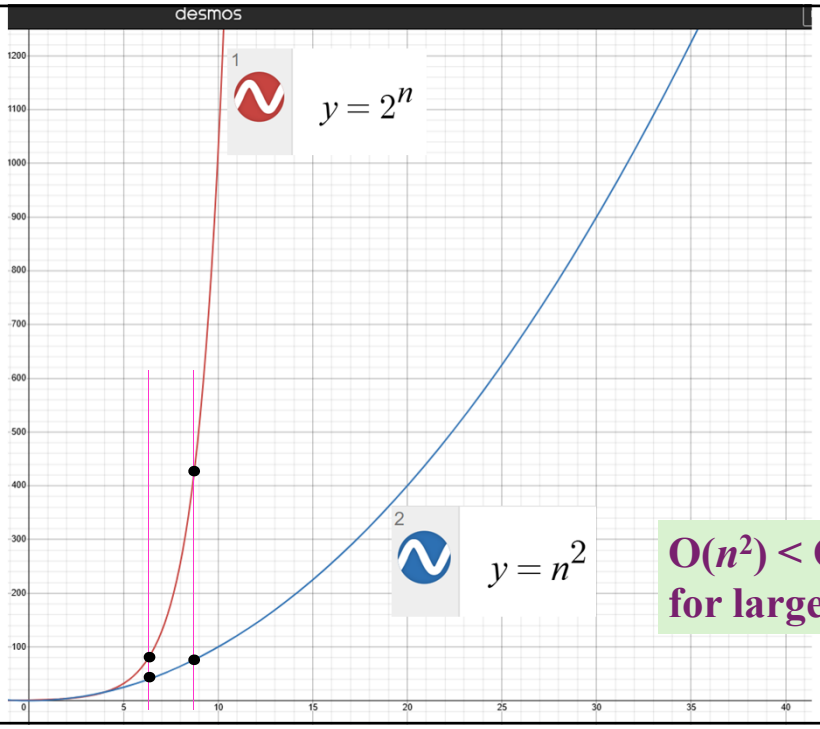
- Naive recursion: recomputes many values ➔ slow
- DP: store previously computed values ➔ fast

In short: Dynamic programming = *solve once, reuse many times*.

53

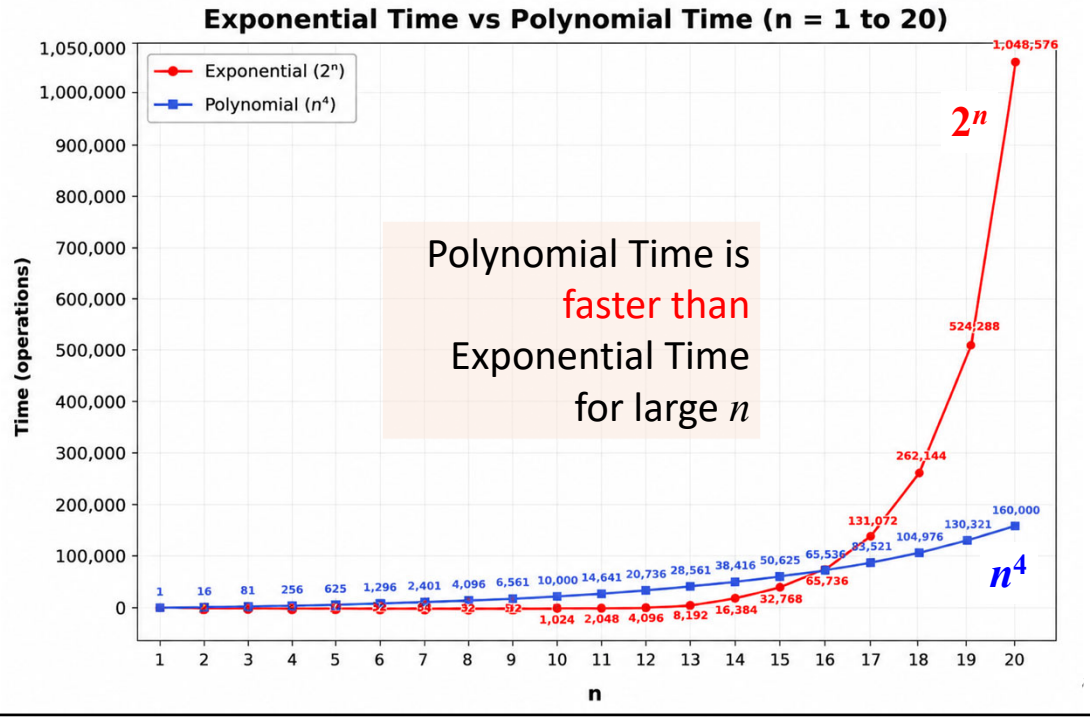
The run time complexity of dynamic programming in this problem is improved to $O(n^2)$, where n is the problem size, and $O(n^2) < O(2^n)$.

54



$O(n^2) < O(2^n)$ for large n

55



Dynamic Programming algorithm is excluded from the first course in C Programming. It is usually not taught in Advanced C Programming course as well.

Dynamic Programming is absolutely a core skill for Programming Olympiads.

Today I will elevate you to this standard.
- A/Prof Tay

57

Dynamic programming (DP)

Example: Fibonacci Sequence

```
#include <stdio.h>
int fib(int n)
{
  int dp [n + 1]; int i;
  if (n <= 1) return n;
  dp[0] = 0; dp[1] = 1;
  for (i = 2; i <= n; i++)  dp[i] = dp[i - 1] + dp[i - 2];
  return dp[n];
}

int main()
{
  int n;
  printf("\n Dynamic Programming (Non Recursive) \n");
  for (n=0;n<10;n++)
    printf("      Fibonacci(%d) = %d\n", n, fib(n));
  return 0;
}
```

Non Recursive !!

9

```
E:\00 S&T-2nd\00 C Programr

 Dynamic Programming (Non Recursive)
       Fibonacci(0) = 0
       Fibonacci(1) = 1
       Fibonacci(2) = 1
       Fibonacci(3) = 2
       Fibonacci(4) = 3
       Fibonacci(5) = 5
       Fibonacci(6) = 8
       Fibonacci(7) = 13
       Fibonacci(8) = 21
       Fibonacci(9) = 34

--------------------------------
Process exited after 0.06756 seconds wit
Press any key to continue . . .
```

The reused array, traditionally, is named **dp** - the storage for **d**ynamic **p**rogramming.

P.T.O.

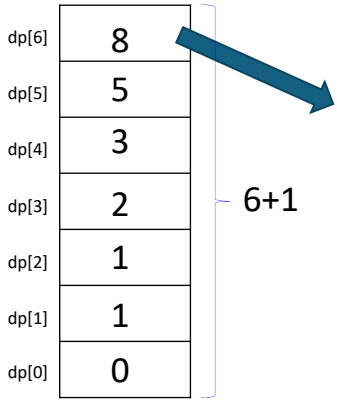
58

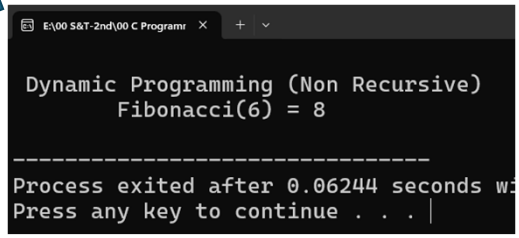
Example: Fibonacci Sequence

Dynamic Programming

6

```
int fib(int n)
{
   int dp [n + 1] ;  int i;

   if (n <= 1) return n;

   dp[0] = 0; dp[1] = 1;

   for (i = 2; i <= n; i++)  dp[i] = dp[i - 1] + dp[i - 2];

   return dp[n];
}
```





solve once, reuse many times.

59

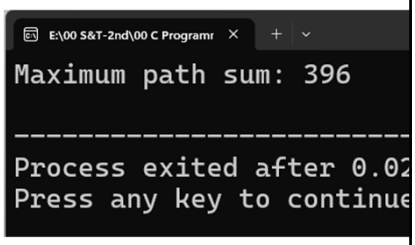
Qn 9b Example: Maximum Sun on a Path on Matrix

Answer:

```
int main()
{
    int n, sum;
    int matrix[MAX_N][MAX_N];

    srand(time(NULL));
    printf("Enter the matrix size (1-10): ");
    scanf("%d", &n);

    if (n < 1 || n > MAX_N) {
        printf("Invalid matrix size.\n");
        return 1;
    }

    randomize_matrix(matrix, n);

    // Print the generated matrix
    printf ("Generated %dx%d Matrix:\n", n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%3d ", matrix[i][j]);
        }
        printf("\n");
    }

    sum = max_total (matrix, n);
    printf("Maximum path sum: %d\n", sum);

    return 0;
}
```

12	42	79	15
60	55	98	65
7	42	32	13
98	64	19	87

Figure 9(c)




```
// Test case: sample 1
int matrix_test[MAX_N][MAX_N] = {
    {12, 42, 79, 15},
    {60, 55, 98, 65},
    {7,  42, 32, 13},
    {98, 64, 19, 87}
};

int sum = max_total(matrix_test, 4);
printf("Maximum path sum: %d\n", sum);
                    // Expect 396
```

60

Qn 9b Example: Maximum Sun on a Path on Matrix

Answer:

```
// Function to compute the maximum sum path using dynamic programming
int max_total(int matrix[MAX_N][MAX_N], int n) {
    int dp[MAX_N][MAX_N];

    dp [0][0] = matrix[0][0]; // Starting position

    // Init first row
    for (int j = 1; j < n; j++)
        dp[0][j] = dp[0][j - 1] + matrix[0][j];

    // Init first column
    for (int i = 1; i < n; i++)
        dp[i][0] = dp[i - 1][0] + matrix[i][0];

    // Dynamic Programming
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = matrix[i][j] + (dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1]);
        }
    }
    return dp[n - 1][n - 1];
}
```

12	42	79	15
60	55	98	65
7	42	32	13
98	64	19	87

Figure 9(c)

MAX_N = 4 in the next test case

P.T.O.

61

Example: Maximum Sun on a Path on Matrix

```
int max_total(int matrix[4][4], int n)
{
  int dp [4][4] , i, j;
  dp[0][0] = matrix[0][0]; // Starting position

  // Initialize first row
  for (j = 1; j < n; j++)
    dp[0][j] = dp[0][j - 1] + matrix[0][j];

  // Initialize first column
  for (i = 1; i < n; i++)
    dp[i][0] = dp[i - 1][0] + matrix[i][0];

  // Dynamic Programming
  for (i = 1; i < n; i++) {
    for (j = 1; j < n; j++) {
      dp[i][j] = matrix[i][j] + (dp[i - 1][j] > dp[i][j - 1] ?
                                 dp[i - 1][j] : dp[i][j - 1]);
    }
  }
  return dp[n - 1][n - 1];
}
```

396

j

matrix:

i

12	42	79	15
60	55	98	65
7	42	32	13
98	64	19	87

j

dp:

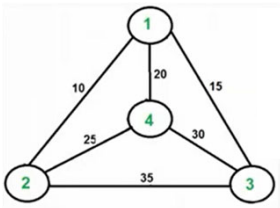
i

12	52	133	148
72	?	231	296
79	169	263	309
177	241	282	396

solve once, reuse many times.

62

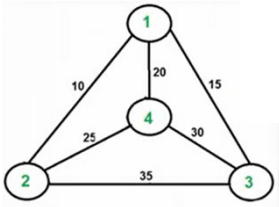
We now increase the challenges to the Olympiads level. A travelling salesman must visit a set of cities and return to the starting point while minimizing the total travel distance. You are given 4 cities, labeled from 1 to 4, and a connection graph where each edge indicates the distance between the two city. Determine the minimum total distance the salesman must travel based on the following requirements: (i) The journey starts at city 1. (ii) Each city is visited exactly once. (iii) The salesman returns to city 1 after visiting all cities.



	1	2	3	4
1	∞	10	15	20
2	10	∞	35	25
3	15	35	∞	30
4	20	25	30	∞

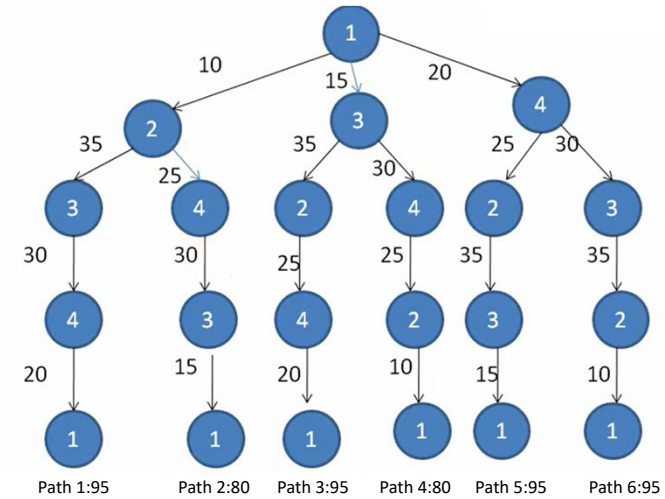
Travelling Salesman Problem

63



	1	2	3	4
1	∞	10	15	20
2	10	∞	35	25
3	15	35	∞	30
4	20	25	30	∞

Travelling Salesman Problem



The shortest paths are Path 2 (1,2,4,3,1) and Path 4 (1,3,4,2,1). But the run time complexity for this method is $O(n!)$.

64

Dynamic Programming Algorithm

Problem: Shortest Path for Travelling Salesman (8 Cities)

A travelling salesman must visit a set of cities and return to the starting point while minimizing the total travel distance. You are given 5 cities, labeled from 0 to 4, and a matrix distance [5][5] where each element represents the distance between two cities.

Write a program to determine the minimum total distance the salesman must travel based on the following requirements: (i) The journey starts at city 0 (index = 0). (ii) Each city is visited exactly once. (iii) The salesman returns to city 0 after visiting all cities.

Input: (i) An integer n = 5.
(ii) A 2D array distance [5][5], where distance [i][j] is the distance from city i to city j.

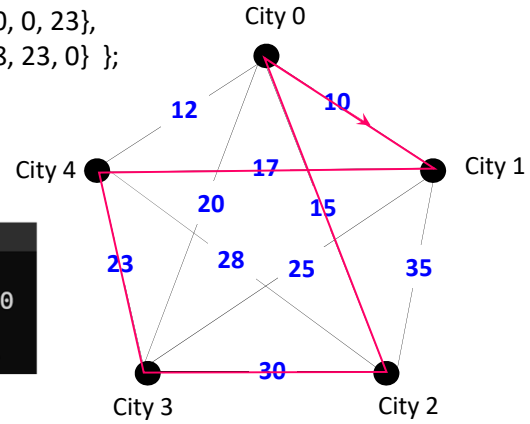
Example: int distance [5][5] = { {0, 10, 15, 20, 12},
{10, 0, 35, 25, 17},
{15, 35, 0, 30, 28},
{20, 25, 30, 0, 23},
{12, 17, 28, 23, 0}};

```
Minimum travel cost: 95
Path: 0 -> 1 -> 4 -> 3 -> 2 --> 0

--------------------------------
Process exited after 0.06306 seconds
 0
Press any key to continue . . .
```

65

Example: int distance [5][5] = { {0, 10, 15, 20, 12},
{10, 0, 35, 25, 17},
{15, 35, 0, 30, 28},
{20, 25, 30, 0, 23},
{12, 17, 28, 23, 0} };

```
Minimum travel cost: 95
Path: 0 -> 1 -> 4 -> 3 -> 2 --> 0

--------------------------------
```



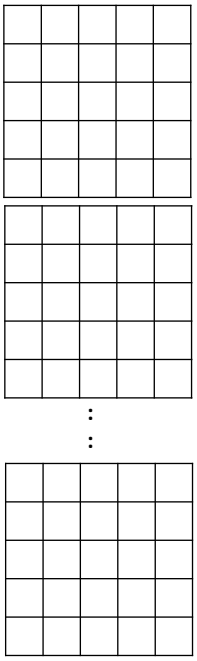
66

```
int distance[N][N] = {
    {0, 10, 15, 20, 12},
    {10, 0, 35, 25, 17},
    {15, 35, 0, 30, 28},
    {20, 25, 30, 0, 23},
    {12, 17, 28, 23, 0}
};
```

The DP mechanism

2^n

All potential subsets of cities

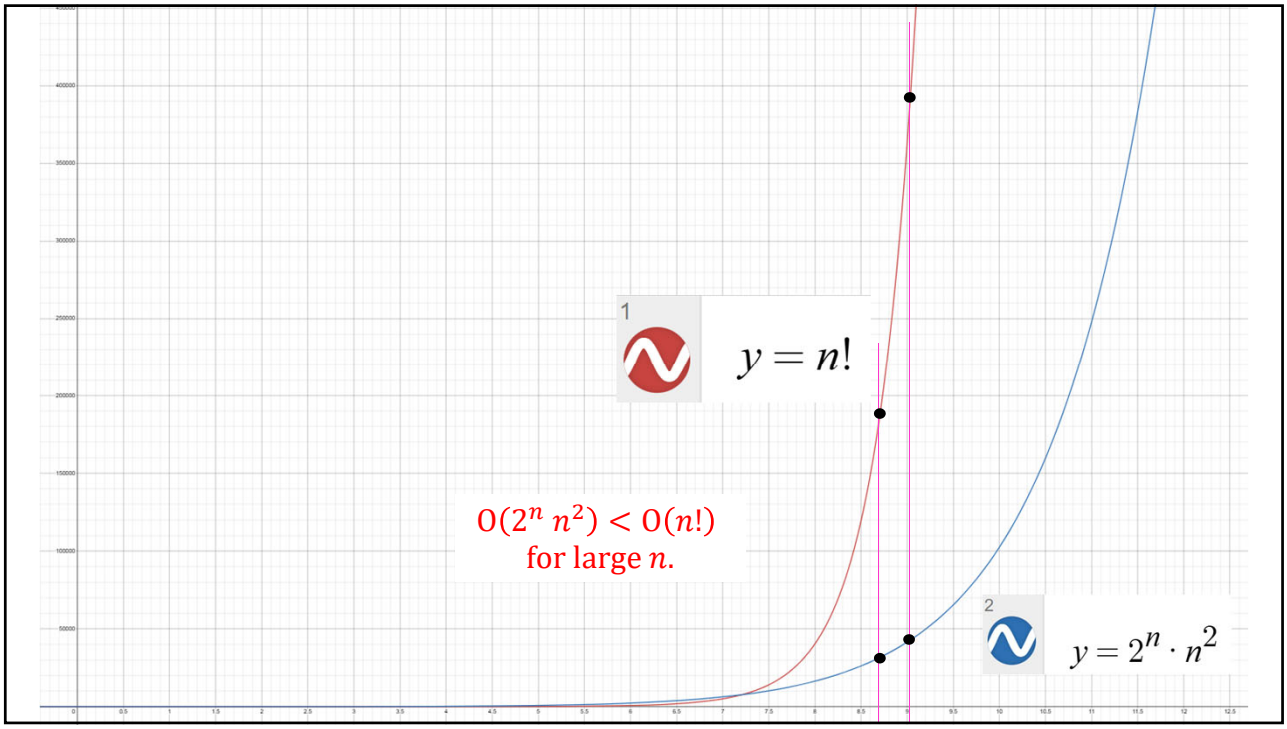


solve once, reuse many times.

Time Complexity is $O(2^n\ n^2)$ for n cities. But,

$O(2^n\ n^2) < O(n!)$ for large n.

67



Dynamic Programming Algorithm for Travelling Salesman Problem

Answer:

```
#include <stdio.h>
#include <limits.h> // for the largest integer

#define N 5
#define num_paths (1 << N)
// 2^5 = 32 - all potential possibilities
// some may not exist

int min (int a, int b) {
   return (a < b) ? a : b;
}
```

```
int main()
 {
   int distance[N][N] = {
      {0, 10, 15, 20, 12},
      {10, 0, 35, 25, 17},
      {15, 35, 0, 30, 28},
      {20, 25, 30, 0, 23},
      {12, 17, 28, 23, 0}
   };

   int result = tsp (distance);

   printf("Minimum travel cost  : %d\n", result);
                                        //95
   return 0;
}
```

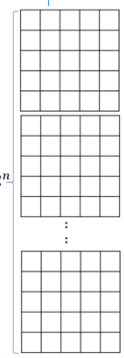
This version does not print the cities in the path.

69

Time Complexity is $O(2^n\, n^2)$ for n cities.

```
int tsp (int distance[N][N])
{
   int dp [num_paths][N], i, j;
   int ans, visited_cities, new_visited_cities;

   // Initialize DP table with large values
   for (i = 0; i < num_paths; i++) {
      for (j = 0; j < N; j++) {
         dp[i][j] = INT_MAX;  // set it to invalid
      }              // 2147483647
   }

   // Starting point: only city 0 visited
   dp[1][0] = 0; // visited cities {00001} at index 0
```

city index: 43210
… 00000: 0 ➔ none visited
… 00001: 1 ➔ city 0 visited
… 01001: 9 ➔ cities 3 and 0 visited
… 00110: 6 ➔ cities 2 and 1 visited



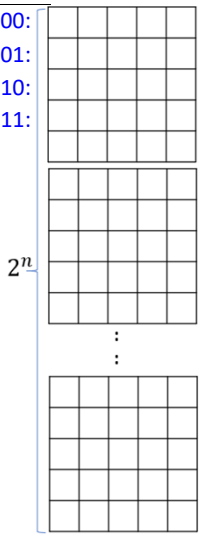

```
// iterate over all subsets of visited cities (bitmasks)
for (visited_cities = 0x01; visited_cities < num_paths;
                                        visited_cities++) {
   for (i = 0; i < N; i++) {
      // If city i is not in visited_cities set, skip
      if (!(visited_cities & (1 << i))) continue;
      // Try to go to next city j
      for (j = 0; j < N; j++) {
         if (visited_cities & (1 << j)) continue; // already visited
         new_visited_cities = visited_cities | (1 << j);
         // only extend if this state is reachable
         if (dp[visited_cities][i] != INT_MAX)
           {   // If this is a valid path that has a basis
               // All states are defined, but only some are
               //      reachable from the start
            dp[new_visited_cities][j] = min(dp[new_visited_cities][j],
              dp[visited_cities][i] + distance[i][j]);  // select the shortest
                                                        //  path from i to j
           }
      } // for j
   } // for i
} // for visited_cities
```

70

From i to j

city index: 43210
… 00000: 0 ➔ none visited
… 00001: 1 ➔ city 0 visited
… 01001: 9 ➔ cities 3 and 0 visited
… 00110: 6 ➔ cities 2 and 1 visited

city index: 43210
… 00000:
… 00001:
… 00010:
… 00011:



if (visited_cities & (1 << j)) continue;
// already visited

E.g., j = 2 (left shift 2 places)

city index: 43210
… 00111
& … 00100

4 = true
already visited ➔ skip

city index: 43210
… 01001
&… 00100

0 = false
not visited yet (new city)
➔ do not skip

// If city i is not in visited_cities set, skip
if (!(visited_cities & (1 << i))) continue;

E.g., i = 3 (left shift 3 places)

city index: 43210
! [… 00101
&… 01000]

!0 = True
not in visited_cities set
➔ skip

city index: 43210
! [… 11001
& … 01000]

!8 = False
in visited_cities set
➔ Do not skip

new_visited_cities = visited_cities | (1 << j);

E.g., j = 2 (left shift 2 places)

… 01001
| … 00100
… 01101

71

// Close the tour (from j to 0), and j can be from any j
// choose the best j (the shortest path up to j)

ans = INT_MAX; // 2147483647 - **assume the worst**

// num_paths - 1 is now 11111 $_{(2)}$, i.e., all cities by now are visited

```
for (j = 1; j < N; j++) {
   if (dp[num_paths - 1][j] != INT_MAX)   // ensure valid set to city j
      ans = min (ans,  dp[num_paths - 1][j] + distance[j][0]);
}                                                  // from j back to 0

return ans;

} //  end int tsp (int distance[N][N])
```

72

The next program …

Full Program with Cities in Optimal Path Printed

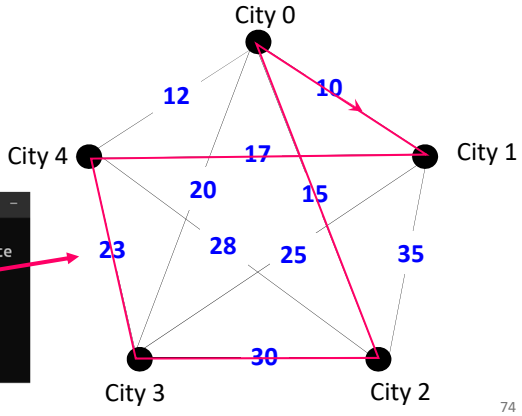
Have to keep track of the parent city of each link.

Cities Printed

```
#include <stdio.h>
#include <limits.h>

#define N 5
#define num_paths (1 << N)

int tsp_with_path(int distance[N][N], int path_out[N + 1])
{
   int      dp [num_paths][N];
   int parent [num_paths][N];  // i to j ➔ parent is i
```

```
Traveling Salesman Problem Solved by Dynamic Programming for Shortest Route
Minimum travel cost: 95
Optimal path: 0  2  3  4  1  0

--------------------------------
Process exited after 0.02082 seconds with return value 0
Press any key to continue . . .
```

```
{0,  10, 15, 20, 12},
{10,  0, 35, 25, 17},
{15, 35,  0, 30, 28},
{20, 25, 30,  0, 23},
{12, 17, 28, 23,  0}};
```



Cities Printed

```
int i, j, visited_cities, new_visited_cities;

int  full_cities, best_last, answer, index, cities_trace,
int  cost, new_cost, current, previous;

// Initialize DP and parent tables
for (visited_cities = 0; visited_cities < num_paths; visited_cities++) {
   for (i = 0; i < N; i++) {
      dp[visited_cities][i] = INT_MAX;  // the longest path length
      parent[visited_cities][i] = -1;       // invalid parent index
   }
}

// Start from city 0 only
dp[1][0] = 0;
```

city index: 43210
...00001

75

Cities Printed

```
// DP over all subsets
for (visited_cities = 0; visited_cities < num_paths; visited_cities++) {
   for (i = 0; i < N; i++) {
      // from i to j
     if (!(visited_cities & (1 << I ))) continue;   // parent not in visited_cities
     if (dp[visited_cities][i] == INT_MAX) continue; // parent unreachable

     for (j = 0; j < N; j++) {

        if (visited_cities & (1 << j)) continue; //  j already visited

        // By now it is OK
        new_visited_cities = visited_cities | (1 << j);  // add j to visited cities ➔ new_visited_cities
        new_cost = dp[visited_cities][i] + distance[i][j];
        if (new_cost < dp[new_visited_cities][j]) {
           dp[new_visited_cities][j] = new_cost;
           parent[new_visited_cities][j] = i; // store previous city (parent of city j)
        }
     }
   }
}
```

76

```
// Find best last city before returning to 0          Cities Printed

full_cities = num_paths - 1;  //   ...0011111  ➔ all cities in
best_last = -1;   // set best last city to be  invalid
answer = INT_MAX;   // set the longest path length

for (j = 1; j < N; j++) {
   if (dp[full_cities][j] != INT_MAX) {   // valid set
      cost = dp[full_cities][j] + distance[j][0];  // choose best j to connect to 0
      if (cost < answer) {
         answer = cost;
         best_last = j;   // best last city set to this j
      }
   }
}   // best_last = 1
```

77

```
// Reconstruct path
 index = N;  // index = 5 for this example
 cities_trace = full_cities;   // ...0011111
 current = best_last;  // 1

 path_out [index--] = 0; // return to start

 while (current != 0 && current != -1) {
    path_out[index--] = current;
    previous = parent[cities_trace][current];
    cities_trace = cities_trace ^ (1 << current);
    current = previous;
 }

 path_out[index] = 0; // start city

 return answer;
}
```

path_out:

0	0
1	2
2	3
3	4
4	1
5	0

```
Traveling Salesman Problem Solved by Dynamic Programming for Shortest Route
Minimum travel cost: 95
Optimal path: 0  2  3  4  1  0

--------------------------------
Process exited after 0.02082 seconds with return value 0
Press any key to continue . . .
```

```
int main()
{                                                    Cities Printed
  int distance[N][N] = {
     {0, 10, 15, 20, 12},
     {10, 0, 35, 25, 17},
     {15, 35, 0, 30, 28},
     {20, 25, 30, 0, 23},
     {12, 17, 28, 23, 0}
  };

  int i, path[N + 1];  // (5+1) city number

  int result = tsp_with_path(distance, path);

  printf("\n Traveling Salesman Problem Solved by
     Dynamic Programming for Shortest Route\n");
  printf(" Minimum travel cost: %d\n", result);

  printf(" Optimal path: ");
  for (i = 0; i <= N; i++) {
     printf("%d  ", path[i]);
  }
  printf("\n");

  return 0;
}
```

78

- End -

79